

Unreliable in Practice? A Comprehensive Study of Errors in LLM-Generated Code

Rodrigo Pato Nogueira^{*†}, Marco Vieira[†], João R. Campos^{*}

^{*}University of Coimbra, CISUC/LASI, DEI
Coimbra, Portugal
jrcampos@dei.uc.pt

[†]University of North Carolina at Charlotte
Charlotte, NC, USA
rpatodec@charlotte.edu, marco.vieira@charlotte.edu

Abstract—Large Language Models (LLMs) are being widely used for coding, with reports indicating that AI now generates an increasing share of production code. Studies show that LLMs can significantly improve developer productivity, yet they still struggle with more complex coding tasks. Just as understanding error modes in human-written code has been central to improving software quality, identifying and characterizing the errors in LLM-generated code is critical for setting realistic expectations and designing mitigation strategies. Prior research has been limited in scope, often focusing on a single language, a small number of problems, or a limited selection of models. As a result, there is still no comprehensive understanding of which errors are common and which are specific to certain models or languages. To address these gaps and develop a deeper understanding of the quality of LLM-generated code, we analyzed a corpus of 86,726 code samples that contained compilation or runtime errors. These samples were generated by seven LLMs across four compiled languages. We classified errors by their underlying causes using an LLM, manually validated these classifications, and performed a comparative analysis. This labeled data is then used to measure error prevalence by model, language, and problem difficulty, to identify common error patterns. Results show that, although error types vary strongly across languages and models, even the largest models frequently make simple mistakes. We also observe that generated code often omits basic input validation or memory-safety checks, which can lead to overflows, resource exhaustion, or other reliability/security issues.

Index Terms—LLMs, Code generation, Errors

I. INTRODUCTION

The use of Large Language Models (LLMs) for code generation has grown rapidly, with up to 30% of Microsoft’s code now AI-generated [1] and estimates suggesting this could reach 95% in the near future [2]. In addition, LLMs have been shown to significantly improve developer productivity, with studies reporting speedups of up to 55.8% [3], [4].

Despite their widespread adoption and productivity benefits, LLMs have significant limitations, showing weak performance on harder problems [5], [6]. These limitations, along with their rapidly increasing use in production environments, make it essential to understand the types of errors these models produce. Such understanding helps developers set realistic

expectations, design mitigation strategies, and craft more effective prompts. However, existing studies on common errors in LLM-generated code remain limited, often covering only one programming language (typically Python) [7]–[10], analyzing a small set of problems [8]–[10] and code samples [7]–[12], and examining code from only one [11], [13] or a few models [12]. These limitations can overlook errors that emerge only at scale or across different programming paradigms. Moreover, the heavy focus on Python restricts the range of errors observed, as its dynamic, interpreted nature prevents many low-level errors, such as those related to manual memory management, which are common in compiled languages.

Just as the systematic study of errors in human-written code has been key to advancing software engineering practices, a comparable analysis of error modes across LLM-generated code and programming languages is essential to understand their limitations and guide the development of more reliable and secure AI-assisted programming systems.

This work addresses these gaps by using a large, multi-model corpus of 86,726 code samples that resulted in compilation or runtime errors. We focus on these error types because they come with explicit diagnostic information (e.g., compiler messages or runtime exceptions), making them easier to identify and analyze consistently at scale. In contrast, incorrect outputs or inefficient solutions require reasoning about the intended behavior of the program, which is more subjective and harder to assess reliably across many samples.

The samples were generated by seven LLMs using both baseline and Chain-of-Thought (CoT) prompting across four compiled languages. We focus on compiled languages as they provide clearer distinctions between error types (compilation/runtime), along with explicit diagnostic messages; in interpreted languages, these distinctions are less clear, making consistent comparison more difficult. The dataset also includes additional samples generated after models received error feedback. Using an LLM-based classifier, all 86,726 compilation and runtime errors were categorized into subtypes based on two taxonomies for errors in LLM-generated code adapted from prior work [11]. We then (1) measure overall and per-model/language error prevalence to identify trends, (2) evaluate which errors are corrected after feedback and which persist, and (3) analyze the impact of CoT prompting on the error distribution. Unlike prior studies, our analysis considers multiple languages, models, problem difficulties, and

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 - Centre for Informatics and Systems of the University of Coimbra, <https://doi.org/10.54499/UID/00326/2025>. This work used computing resources provided through the Google Cloud Research Credits program with the award GCP19980904 and AWS Cloud Credits for Research provided by Amazon Web Services.

prompting strategies, as well as iterative feedback, providing a more comprehensive characterization of LLM code-generation errors. To the best of our knowledge, this is the first large-scale, cross-language study spanning multiple languages, models, prompts, and iterative feedback.

Results show that the kinds of errors made by LLMs differ across programming languages and models, reflecting differences in the amount and quality of training data available for each language, as well as the challenges specific to each language. Yet, despite these differences, all models still produce a substantial number of fundamental mistakes, such as undeclared variables, missing imports, or simple type mismatches. In addition, we find that the generated code often lacks basic input checks and memory-safety safeguards (e.g., unbounded reads or unsafe memory operations), which can ultimately compromise system dependability and introduce vulnerabilities. Notably, these issues persist even in bigger commercial models such as GPT-4.1-mini, indicating that such fundamental reliability problems are not confined to smaller open-source models. These types of issues are particularly concerning, as they are more difficult to identify and have the potential for causing system failures or security vulnerabilities, and therefore have a significantly higher impact than purely functional mistakes. This highlights critical limitations that may lead to unreliable and insecure systems. In this study, we evaluate LLMs in a competitive-programming setting without IDE assistance, compiler or runtime feedback beyond standard diagnostics, or human-in-the-loop intervention; under these conditions, the results indicate that current models are still far from being suitable for reliable standalone code generation, especially in dependability- or security-sensitive settings.

To support reproducibility, we make publicly available the full experimental pipeline, including the code used to prompt the models, perform error classification, and compute all reported metrics, together with the resulting per-sample error labels, at: <https://tinyurl.com/UnreliableInPractice>.

In summary, the main contributions of this paper are:

- The largest analysis of errors in LLM-generated code to date, covering 86,726 compilation and runtime errors across seven models and four compiled programming languages.
- A detailed characterization of error types, showing how they differ across models, languages, and problem difficulty.
- The first evaluation of the impact of iterative feedback across multiple models and languages, identifying which error categories are more or less likely to be corrected.

The remainder of this paper is organized as follows. Section II presents the background and related work, Section III describes the experimental setup, Section IV presents and discusses the results, and Section V addresses the threats to validity. Finally, Section VI concludes the study.

II. BACKGROUND AND RELATED WORK

Recently, there have been significant advances in LLMs, and these models are now widely used in everyday developer workflows for a range of tasks [14], [15]. One of the clearest examples is text-to-code generation: given a natural-language

TABLE I: Related Work

Paper	Langs.	Problems	Samples	Models	Prompts
[7]	1	1164	3084	3	4
[8]	1	164	557	6	1
[9]	1	164	?	3	1
[10]	1	230	333	3	2
[12]	2	2268	?	2	1
[13]	4	2033	4191	1	1
[11]	5	728	728	1	1
Our study	4	1651	86,726	7	2

description, modern models can produce runnable implementations. LLMs trained on large code corpora learn idioms, common APIs, and repository conventions, enabling them to generate functional code with little or no manual editing [16]–[19]. These capabilities are already reshaping how developers prototype, complete, and test code in practice [2]. However, LLMs still frequently make mistakes, ranging from obvious issues, such as calling functions before declaring them, to subtler but critical problems, including missing edge cases or generating code vulnerable to issues like integer overflows or unsafe memory use [6], [11]. These latter errors are particularly concerning, as they are harder to detect and may not affect functional behavior under normal conditions, while still compromising reliability and security. Understanding these errors is essential for safely and effectively relying on LLMs.

Program failures can happen at distinct places: code can fail to compile, crash or raise errors at runtime, produce incorrect functional outputs (failing unit tests), or violate non-functional constraints such as excessive runtime or memory use. Which of these errors appears most often depends on the implementation language. Low-level languages like C/C++ expose memory-safety and undefined-behavior faults (e.g., buffer overflows, use-after-free), whereas managed or dynamic languages such as Java and Python largely prevent or abstract away these issues through runtime checks and automatic memory management [20]. As a result, errors in these languages more often manifest as API misuse, logic errors, or missing corner cases rather than low-level memory corruption. Understanding exactly where and why code fails has long been a focus of traditional software engineering, with taxonomies such as Orthogonal defect classification (ODC) [21] having been developed to categorize faults and guide diagnostics.

Several recent efforts have measured how well LLMs perform at code generation, but they differ in what they measure and how they report results. Many benchmark-style studies focus on functional correctness, evaluating models based only on whether the generated code passes a set of unit tests or not [5], [19], [22]. By contrast, CodeXGLUE [23] emphasizes reference-based similarity metrics to quantify how close a generated solution is to a canonical implementation [23]. Although these studies provide a good idea of how LLMs perform in code generation, they do not give users insight into what went wrong or why the generated code was incorrect.

A different line of work goes beyond aggregate correctness to study what goes wrong in generated programs. These studies rely on manual inspection to build taxonomies of errors in LLM-generated code and identify common failure modes, providing more actionable insights than simple “pass/fail” metrics [7]–[13]. However, existing error analyses exhibit several

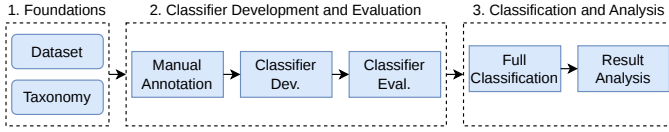


Fig. 1: Methodology

limitations (Table I). Most focus on a single programming language, typically Python, limiting the range of observable errors and generalizability [7]–[10]. Python’s dynamic execution model masks entire classes of failures related to memory management and undefined behavior, and even extensions to languages such as Java abstract away many system-level error modes common in lower-level languages [12]. In addition, prior work often relies on either a small number of problems or a limited number of samples per problem [7]–[11], reducing statistical power. Finally, most studies [7], [9]–[13] evaluate only one or a few models, making it difficult to distinguish model-specific artifacts from general error patterns shared across models and architectures. They also rarely examine how prompting choices affect error distributions, leaving an important dimension of variability underexplored.

Our study analyzes 86,726 runtime or compilation errors across seven models, two prompting techniques (baseline and CoT prompting), four compiled programming languages, and 1,651 problems. This scale and diversity let us (i) quantify how error types vary across languages, (ii) separate model-specific quirks from broader trends, (iii) assess the impact of prompting on error behavior, and (iv) measure both compilation and runtime errors with statistically meaningful sample sizes.

III. EXPERIMENTAL SETUP

Our study follows the three-stage methodology shown in Figure 1. First, foundations are established by selecting a dataset of problem–code–error pairs and an error taxonomy. Then, a subset of the dataset is manually annotated according to the taxonomy and used to assess different LLM-based solutions to identify a suitable approach for automated error classification. Finally, in the large-scale classification and analysis stage, the selected solution classifies all samples, and the resulting error distribution is analyzed.

A. Foundations

The first stage establishes the foundations of the study by defining the dataset of {code–diagnostic–test–summary} samples and adopting an existing error taxonomy to guide the subsequent analysis. We intentionally focus on an unassisted setting to capture the natural error tendencies of current models, without external tooling or human intervention.

1) *Dataset*: The dataset used for this study is based on the PROBE dataset [24]. This dataset is made up of 1,651 programming-contest-style problems from IBM’s Project CodeNet dataset [25], grouped into four difficulty levels. Each problem is accompanied by a set of unit tests and reference solutions and contains a large set of LLM-generated solutions. These solutions were generated by six models (temperature 0.6): four smaller open-source models (Qwen2.5:14b, Qwen2.5-Coder 7B and 14B, and DeepSeek-Coder-v2) and

two larger proprietary models (GPT-4.1-mini and Gemini-2.0-Flash). Each model generated five solutions per problem under two prompting strategies (0- and 1-shot) and in four programming languages (C++, Java, C, and Rust), resulting in 396,240 samples. Each sample consists of an initial generation and, if incorrect, up to two additional generations obtained through feedback after testing.

This dataset was chosen due to three key factors. First, it includes a large number of problems spanning multiple difficulty levels, enabling analysis across a wide range of programming challenges. Second, it provides a substantial volume of LLM-generated solutions produced by a diverse set of models, allowing for robust cross-model comparisons of compilation and runtime errors. Finally, the inclusion of a feedback-driven generation process (i.e., iterative re-generation after testing) enables the analysis of how such feedback mechanisms influence error patterns and solution diversity.

To study the impact of prompting, we extend the dataset by generating additional solutions using CoT prompting, while keeping all other aspects of the generation process unchanged. In contrast to the original setup, the CoT setting uses single-pass generation without feedback, allowing comparison between baseline and CoT prompting without the confounding effects from feedback-based correction. To provide a more up-to-date picture of model behavior, we further extend both the baseline and CoT settings with GPT-oss:120b, a more recent and larger open-weight model.

The original study reported identical functional correctness, similarity, and code quality metrics for both 0-shot and 1-shot settings across all models and languages. However, the results indicate that, in this context, the inclusion of examples did not impact performance. Therefore, for conciseness, we focus only on the 0-shot and the new CoT settings. We analyze the samples with compilation or runtime errors. Across the two prompting techniques, a total of 52,669 samples failed to compile: 9,953 in C++, 6,996 in Java, 12,304 in C, and 23,416 in Rust (Rust enforces stricter compile-time checks, resulting in more frequent compilation errors). Additionally, 34,057 samples resulted in at least one runtime error during unit testing: 5,605 in C++, 8,279 in Java, 9,135 in C, and 11,038 in Rust.

We focus exclusively on compiled languages (C++, Java, C, and Rust), excluding interpreted languages due to fundamental differences in how errors manifest and are classified. In interpreted settings, distinct failure modes are often collapsed into runtime errors. For example, an undeclared variable may trigger a compilation error in compiled languages, but appear as a runtime error in interpreted ones, despite differing underlying causes. This leads to inconsistencies in error categorization and hinders meaningful cross-language comparisons. Furthermore, we restrict our analysis to compilation and runtime errors, excluding issues such as incorrect outputs or timeouts. Unlike compilation and runtime failures, which provide explicit diagnostic signals, these errors require reasoning about program semantics and intended behavior, making their classification inherently more ambiguous. Focusing on observable error

types enables more robust and consistent analysis.

2) *Taxonomy*: Our goal is not to propose a new taxonomy, but to operationalize and validate existing taxonomies for LLM-generated code in a cross-language, large-scale setting. To this end, we employ two complementary taxonomies to capture distinct failure modes: one for compilation errors and another for runtime errors. Both are based on the schemata proposed by Liu et al. [11], derived from the manual analysis of 728 LLM-generated code samples for LeetCode problems.

We chose not to rely on traditional defect taxonomies such as ODC, as these are primarily designed for human-written software and operate at a coarser level of abstraction. While such taxonomies can capture underlying defect causes that may lead to compilation failures (e.g., undeclared variables), they do not explicitly distinguish or categorize compilation errors as first-class outcomes. This limitation is particularly relevant in the context of LLM-generated code, where compilation errors are both frequent and structurally distinct. LLMs often produce error patterns such as missing imports or invalid type annotations, that, in conventional development workflows, would often be caught and corrected immediately through IDE support, compiler feedback, or manual review.

While we adopted the structure proposed by Liu et al. [11] as our starting point, several targeted adjustments were made to improve its generality and applicability across multiple languages. Specifically, we refined the taxonomy to remove environment-specific categories tied to the original platform (i.e., LeetCode), merge overlapping definitions, and adjust categories that were either too narrow or ambiguous. In addition, a small number of additional classes were introduced to account for error types not covered in the original schema when extending the taxonomy across languages. The key changes are summarized below.

- 1) **LeetCode-specific labels removed**. Labels referencing LeetCode artifacts (e.g., method name templates) do not apply to our setting and were removed.
- 2) **“Syntax Error” removed**. This broad class overlapped with more specific categories such as “Label error” and “Redefinition” and was removed to avoid redundancy.
- 3) **“Invalid operators” and “Incompatible parameter types” merged**. Both capture type-related mismatches, one in expressions, and the other in function arguments. These were consolidated into a single “Incompatible parameter types” category to improve clarity and cohesion.
- 4) **“Missing delimiters” removed**. This fine-grained syntactic class (e.g., missing commas or brackets) operates at a lower level of abstraction than the remaining categories and was removed to maintain consistency.
- 5) **“Missing imports” category added**. Omitting required imports was a recurring error mode in our corpus not covered by the original taxonomy.
- 6) **Java-specific “Missing Public Class” added**. Unlike LeetCode, standalone Java files require a public class to compile; we added an explicit category to capture this.
- 7) **Rust-specific errors added**. Two categories for Rust-specific compilation errors were introduced: (i) “Ownership

and lifetime errors” for violations of Rust’s ownership and borrowing rules, and “Trait or type-bound errors” for unsatisfied trait constraints or failed type inference.

- 8) **“Other” bucket added**. Covers infrequent or unexpected compilation errors not fitting any defined category.

For the Runtime Errors, the following changes were made:

- 1) **“Heap-buffer-overflow” category replaced by broader “Out-of-memory” definition**. The original category overlapped conceptually with “Out-of-bound” and did not generalize to managed languages. To improve clarity and cross-language applicability, we redefined “Out-of-memory” to cover all memory allocation failures and extended “Out-of-bound” to all invalid indexing operations.
- 2) **“Other” bucket added**. Covers infrequent or unexpected runtime errors not fitting any defined category.

The resulting taxonomies are designed to support consistent error classification across multiple programming languages while preserving language-specific distinctions where necessary (e.g., ownership errors in Rust or missing public classes in Java). This enables a systematic, large-scale empirical analysis of LLM-generated code errors across languages, rather than the introduction of a new taxonomy. The complete taxonomies are shown in Tables II and III.

B. Classifier Development and Evaluation

The goal of this stage is to design a solution for mapping each {code + diagnostic + test-summary} sample to taxonomy labels. While manual classification is preferable for quality, it is infeasible at the scale of our dataset; therefore, we build and evaluate automated, LLM-based classification solutions on a human-labeled subset and then select the best-performing solution for large-scale application.

1) *Manual annotation*: A labeled evaluation set is required both to measure model performance and to validate the taxonomies. For each error family (compilation and runtime), we randomly sampled 300 examples, stratified by language (75 per target language). Each sampled example consists of the generated code, the associated error message, and, for runtime errors, the unit-test input that triggered the error. Although this approach does not ensure coverage of all taxonomy classes, it preserves the original error distribution within each language. The following annotation protocol was followed:

- 1) **Independent labeling**. Two experts independently label each example according to the taxonomy.
- 2) **Agreement**. Labels are compared and inter-annotator agreement is measured (see metrics below). In case of disagreement, reviewers presented their points of view and discussed until an agreement was reached.

The inter-annotator agreement was measured using the Jaccard index for compilation errors (as the categories are not mutually exclusive) and Cohen’s κ for runtime errors (as the categories are mutually exclusive). For compilation errors, a Jaccard index of around 0.8 was obtained, with the observed differences between annotators mainly stemming from a small number of recurring errors that were interpreted

TABLE II: Compilation Error Classification

Error Type	Description
Label error	A label can only be part of a statement, and a declaration is not a statement.
Redefinition	A symbol has been defined in multiple places.
Function declaration error	Function is invoked before declaration.
Undeclared variable	Variable is referenced or used in a program without being previously declared or defined.
Constant function	Function (or method) is generated with an empty body.
Use an undeclared function	Generated code snippet uses undeclared and undefined functions.
Incompatible parameter types	Mismatch between the expected parameter type and the actual argument type.
Uninitialized variable	Variable is being declared and initialized, but not given a valid initial value before being used.
Error of #include	Indicated file cannot be compiled (e.g., trying to include a file that does not exist).
No attribute	Attempt to access a field or method that does not exist in a defined struct/class type.
Missing import or #include	A function or variable from a package is called without the package being imported/included.
Missing Public Class	No public class in the code.
Ownership and lifetime error	Violations of Rust’s ownership, borrowing, or lifetime rules.
Trait or type-bound error	Required trait implementations/type constraints not satisfied, or type inference fails due to ambiguous/missing information.
Other	Miscellaneous compilation errors not covered by other categories.

differently. For runtime errors, Cohen’s κ was around 0.62, indicating substantial agreement [26], especially considering the wide variety of classes and the fact that many samples were inherently complex to annotate.

2) *Classifier Development*: We design a zero-shot LLM-based classifier to map {code + diagnostic + test-summary} samples to taxonomy labels, enabling scalable analysis of compilation and runtime errors across a large corpus of generated programs. The emphasis at this stage is on identifying a sufficiently reliable and stable configuration for automated labeling, rather than on conducting a comparative evaluation of LLMs. We evaluate **GPT-4.1-mini** with direct and CoT prompting, and **Gemini-2.0-Flash** with direct prompting only. Gemini-2.0-Flash was not evaluated with CoT because it already performed substantially worse than GPT-4.1-mini under direct prompting. Given that gap, CoT was unlikely to make it the best-performing classifier, and the additional inference cost was not warranted. All models are evaluated with temperature 0, top- k of 50, and top- p of 1.0 to promote deterministic and reproducible outputs, and are accessed via their respective APIs (OpenAI and Google Vertex AI, respectively). While strict determinism is not guaranteed for commercial LLMs, this configuration produced consistent outputs across runs.

3) *Classifier Evaluation*: Classifiers are evaluated against the manually annotated dataset to identify the best-performing model. Evaluation is based on strict accuracy: a prediction is considered correct only if the set of errors exactly matches the human annotations. We do not report macro-averaged precision or recall, as these metrics proved unstable due to the rarity of several error classes. Results for compilation and runtime error classification are summarized in Table IV.

For compilation errors, GPT-4.1-mini achieved the best overall performance, with accuracies of 0.91 for C++, 0.91 for Java, 0.84 for C, and 0.91 for Rust without CoT. The highest accuracy (0.92) was achieved with C++ and CoT. Gemini-2.0-Flash consistently underperformed, particularly for C (0.59). For runtime errors, GPT-4.1-mini again outperformed Gemini-2.0-Flash across all languages. Without CoT, accuracies ranged from 0.81 to 0.91. CoT prompting yielded marginal improvements for Java and C, but slightly reduced performance for C++ and Rust. Given the negligible gains and additional inference cost, GPT-4.1-mini (no CoT) was selected for both compilation and runtime error classification.

Table V reports per-class recall and precision (with support) for the selected model (GPT-4.1-mini) on compilation and runtime errors. Only classes with at least three samples in at least one language are included. Results show strong performance on common error classes, while rarer ones exhibit greater variability due to limited representation. The model also shows lower recall for the “Other” category, indicating difficulty identifying this residual class. Several classes are not observed for certain languages. As the dataset was built by randomly sampling 75 errors per language, their absence suggests they are rare and have limited impact on aggregate trends. For runtime errors, results indicate a bias toward predicting “Out-of-Bound” errors, reducing recall for classes such as “Out-of-memory” and “Recursion error”. Manual analysis revealing that this was due to the model having a tendency to default to an “Out-of-Bound” classification when a segmentation fault is observed, and thus error information is limited.

Overall, while strict accuracy ranges between 0.81 and 0.92, the per-class analysis shows that remaining errors are concentrated among a small subset of classes and follow consistent patterns. To quantify the impact of these errors on the final results, we applied a confusion-matrix-based correction: for each class, the observed corpus count is scaled by the ratio of precision to recall in the manually annotated evaluation set, yielding a bias-corrected proportion. Although the full per-class breakdown is omitted here for space, this correction did not alter any of the overall trends reported in Section IV.

C. Classification and analysis

Having selected an LLM-based solution to map generated samples to taxonomy labels, we apply it to the entire dataset and conduct a comprehensive analysis of model error behavior.

1) *Full classification*: The classifier is run over all samples.

2) *Analysis*: The results are analyzed in detail. We start by examining the overall distribution of error types for baseline prompting and how it varies with factors such as programming language, model, and problem difficulty. Next, we study how iterative feedback influences each error type. Finally, we analyze the impact of CoT prompting on the error distribution.

To assess whether the observed differences in error frequencies are statistically significant, we use two-proportion z-tests. We chose z-tests because our sample sizes are large, making

TABLE III: Runtime Error Classification

Error Type	Description
1. Numeric overflow	The result of an arithmetic operation exceeds the maximum value that can be represented by a given numeric data type.
2. Out of memory	The program fails to allocate memory due to insufficient heap or system resources.
3. Undefined behavior	The execution of a certain code statement is not defined or predictable, potentially leading to unexpected runtime errors or crashes (e.g., left shift of negative value).
4. Out of bounds	Accessing a data structure at an index or memory address that lies outside its valid range.
5. Constant function	Function (or method) is generated with an empty body.
6. Null pointer dereference	Accessing a memory location or object that is explicitly null due to a programming logic error.
7. Type error	An operation is performed on an object of an incompatible type.
8. Value error	A method or operation receives a valid type of input. Still, the value itself is not suitable or within the expected range for the specific operation (e.g., calling <code>.max()</code> on an empty array in Java).
9. Heap use-after-free	Use a memory block in the heap after it has been deallocated (freed).
10. Recursion error	A function/method calls itself recursively without reaching a base case or termination condition.
11. Uninitialized variable	Variable is being declared and initialized, but not given a valid initial value before being used.
12. Undeclared variable	Variable is referenced or used in a program without being previously declared or defined.
13. No attribute	The accessed member does not exist in the corresponding data structure.
14. Divided by zero	Attempt to divide a number by zero.
15. Incorrect input processing	Errors caused by wrong assumptions about the input format.
-1. Other	Miscellaneous runtime errors not covered by other categories.

TABLE IV: Exact match accuracy of the classifiers

Model	Compilation				Runtime			
	C++	Java	C	Rust	C++	Java	C	Rust
GPT-4.1-mini	0.91	0.91	0.84	0.91	0.81	0.91	0.83	0.84
Gemini-2.0-Flash	0.80	0.88	0.59	0.79	0.71	0.80	0.53	0.72
GPT-4.1-mini (CoT)	0.92	0.85	0.83	0.81	0.80	0.92	0.84	0.83

```

1 #include <iostream>
2 #include <bitset>
3 int main() {
4     uint32_t a, b;
5     ...

```

Listing 1: Example of a missing import error

the normal approximation appropriate, and because the test is simple, standard, and directly interpretable.

IV. RESULTS AND DISCUSSION

In this section, we present a comprehensive analysis of the classification results, aiming to characterize the types and distributions of errors produced by the models. We first examine compilation errors, followed by runtime errors. For each category, we analyze its distribution and explore how programming language, model, problem difficulty, and prompting influence these patterns. Due to space constraints, all tables in this section display only the results for labels with a frequency of above three percent in any column (i.e., any language for the language tables and any model for the model tables).

A. Compilation Errors

Table VI presents the distribution of compilation errors per programming language (left) and per problem difficulty (right) for the baseline prompt. Each cell reports both the number of errors in both absolute and relative terms (within that language or difficulty level). Two dominant error types account for most compilation errors across all languages: “Missing import” and “Incompatible parameter types”, followed by “Undeclared variable”. Rust shows a distinct pattern, with 16.7% of errors related to “Ownership and lifetime” and 6.1% to “Trait/type bound” issues. Examples of the two most common compilation error types are shown in Listings 1 and 2. The first example is a code sample generated by GPT-4.1-mini, which fails to compile because `uint32_t` is used without including `<stdint>`. The second example illustrates an “Incompatible parameter types” error in Java, where a primitive variable is combined with a `BigInteger` using the modulus operator, leading to a type mismatch.

Two additional insights can be highlighted. First, the total number of errors labeled as “Other” is low, indicating that the taxonomy captures the vast majority of issues. Although it is

unlikely the model captured all errors that should belong to this category, the recall and accuracy for this class suggest that the real number is still small. Additionally, when examining the language-specific error classes (“Missing Public class” for Java and “Ownership and lifetime error” and “Trait or type bound error” for Rust), we observe that the models correctly attribute these classes only to examples of the respective language. Across the dataset, “Missing Public Class” was assigned 594 times in total, 593 of which (99.8%) were in Java. Likewise, the combined total for “Ownership and lifetime error” and “Trait or type bound error” is 2,953 classifications, of which 2,949 (99.9%) belong to Rust. These results indicate that the models correctly map language-specific compiler diagnostics to the corresponding taxonomy labels.

When we break down the distribution per language, some clear trends emerge. For C and C++, “Missing import” is by far the most common classification, accounting for 56.6% of C++ compilation errors and 35.5% of C compilation errors. The same cannot be said for the other two languages, as “Missing import” accounts for only 8.1% of compilation errors in Java and 20.5% in Rust. All pairwise differences between languages are statistically significant (two-proportion z-tests with Bonferroni correction and $\alpha=0.05$). These differences can be explained by how standard libraries are designed across these languages. C and C++ require explicit includes for nearly all functionality (even basic operations like I/O need

```

1 public class BenchPhotoProbability {
2     private static final BigInteger MOD = new BigInteger("1000000007");
3     public static void main(String[] args) {
4         ...
5         long p = 0, q = 0, r = 0;
6         ...
7         for (int i = 0; i < N; ++i) {
8             if (s.charAt(i) == 'X') {
9                 ...
10                p %= MOD;
11                q %= MOD;
12                r %= MOD;
13                ...

```

Listing 2: Example of a incomp. param. types error

TABLE V: Recall/Precision and support for GPT-4.1-mini across Compilation (above) and Runtime (below) errors

Category	-1	3	4	6	7	13	14	15	16
C++	0.50/0.67 (4)	0.50/1.00 (2)	1.00/0.91 (10)	1.00/0.25 (1)	0.89/0.94 (19)	0.95/0.98 (44)	0.00/0.00 (0)	0.00/0.00 (0)	0.00/0.00 (0)
Java	0.69/1.00 (13)	0.00/0.00 (0)	0.88/1.00 (17)	1.00/1.00 (1)	0.96/1.00 (28)	1.00/1.00 (8)	1.00/0.75 (6)	0.00/0.00 (0)	0.00/0.00 (0)
C	0.25/0.50 (4)	0.60/1.00 (5)	0.89/1.00 (9)	0.90/0.75 (10)	0.96/0.87 (27)	1.00/0.96 (25)	0.00/0.00 (0)	0.00/0.00 (0)	0.00/0.00 (0)
Rust	0.25/1.00 (4)	0.00/0.00 (1)	1.00/1.00 (1)	1.00/0.50 (1)	0.95/0.97 (37)	1.00/1.00 (18)	0.00/0.00 (0)	1.00/0.83 (19)	0.83/0.83 (12)

Category	-1	1	2	3	4	10	14	15
C++	0.00/0.00 (1)	0.80/1.00 (5)	0.67/0.67 (6)	0.80/0.80 (5)	0.97/0.78 (33)	0.69/1.00 (13)	1.00/1.00 (5)	0.25/0.50 (4)
Java	0.00/0.00 (0)	0.38/1.00 (8)	1.00/0.50 (1)	0.00/0.00 (0)	1.00/0.97 (31)	0.92/1.00 (12)	1.00/1.00 (1)	0.95/0.83 (21)
C	0.00/0.00 (3)	1.00/1.00 (2)	0.81/0.88 (26)	1.00/1.00 (2)	0.94/0.83 (32)	0.67/0.67 (3)	1.00/1.00 (3)	0.00/0.00 (0)
Rust	0.00/0.00 (3)	1.00/1.00 (29)	0.00/0.00 (0)	0.00/0.00 (0)	1.00/0.64 (14)	1.00/1.00 (1)	0.00/0.00 (0)	0.68/0.95 (28)

TABLE VI: Compilation Error Distribution per language (left) and problem difficulty (right) (baseline prompt)

	C++	Java	C	Rust	Diff. 0	Diff. 1	Diff. 2	Diff. 3
Undeclared var.	410 (6.1%)	566 (15.7%)	713 (10.4%)	490 (3.8%)	330 (4.4%)	854 (7.6%)	777 (9.1%)	218 (7.9%)
Use undec. func.	454 (6.8%)	64 (1.8%)	1150 (16.8%)	177 (1.4%)	447 (5.9%)	774 (6.8%)	461 (5.4%)	163 (5.9%)
Inc. param. types	1358 (20.3%)	1551 (43.0%)	1497 (21.9%)	5616 (43.4%)	2209 (29.4%)	3916 (34.6%)	2954 (34.7%)	943 (34.0%)
Uninitialized var.	25 (0.4%)	63 (1.7%)	333 (4.9%)	79 (0.6%)	136 (1.8%)	214 (1.9%)	103 (1.2%)	47 (1.7%)
No attribute	238 (3.5%)	179 (5.0%)	188 (2.7%)	438 (3.4%)	224 (3.0%)	384 (3.4%)	322 (3.8%)	113 (4.1%)
Missing import	3795 (56.6%)	292 (8.1%)	2432 (35.5%)	2656 (20.5%)	2924 (38.9%)	3318 (29.3%)	2219 (26.1%)	714 (25.7%)
Miss. Pub. Class	0 (0.0%)	593 (16.4%)	0 (0.0%)	1 (0.0%)	116 (1.5%)	181 (1.6%)	234 (2.8%)	63 (2.3%)
Own. and life.	2 (0.0%)	0 (0.0%)	0 (0.0%)	2165 (16.7%)	618 (8.2%)	753 (6.7%)	592 (7.0%)	204 (7.4%)
Trait/type bound	0 (0.0%)	2 (0.1%)	0 (0.0%)	784 (6.1%)	190 (2.5%)	295 (2.6%)	226 (2.7%)	75 (2.7%)
Other	107 (1.6%)	129 (3.6%)	201 (2.9%)	334 (2.6%)	128 (1.7%)	255 (2.3%)	279 (3.3%)	109 (3.9%)
Total	6706 (100%)	3611 (100%)	6850 (100%)	12942 (100%)	7515 (100%)	11311 (100%)	8508 (100%)	2775 (100%)

#include <stdio.h> or #include <iostream>). Java and Rust, however, take a different approach: Java automatically imports Java.lang.*, while Rust’s prelude brings common types and traits into scope by default. As a result, LLMs generating Java or Rust code is less likely to omit necessary imports because fewer explicit import statements are needed to begin with.

The “Incompatible parameter types” error was more prevalent in Java (43.0%) and Rust (43.4%) than in C++ (20.3%) and C (21.9%). The differences are statistically significant between all models pairs except between C++ and C, and between Java and Rust (two-proportion z-tests with Bonferroni correction and $\alpha=0.05$). In Java, this higher proportion is primarily a relative effect: the absolute number of errors in this category (1,551) is comparable to that in C++ (1,358) and C (1497), so the percentage difference primarily reflects the much lower number of missing import errors in Java. In contrast, Rust exhibits a substantially higher absolute number of incompatible parameter type errors (5,555), indicating a genuine increase rather than a relative effect. This observation is consistent with prior work showing that LLMs struggle with Rust in code-related tasks such as code translation [27]. While the tasks differ, both findings suggest that Rust poses particular challenges for LLMs, potentially due to its strict type system and lower representation in training data.

When focusing on the per-problem difficulty distribution, we observe that, although the overall shifts in error distributions are not large, the distribution varies significantly across difficulty levels (chi-square tests with Bonferroni correction and $\alpha=0.05$). Some notable trends can be observed. The relative frequency of Missing imports was higher in easier problems, dropping from 38.9% in difficulty 0 to 25.7% in difficulty 3. Conversely, incompatible parameter types and use of undeclared functions showed the opposite pattern, with incompatible parameter types increasing from 29.4% to

34.7% between difficulty levels 0 and 2, while undeclared variables rose from 4.4% to 9.1%. This is due to more complex problems involving more variables and function calls, increasing the likelihood of parameter mismatches or undeclared references. In simpler problems, where fewer functions and variables are needed, these errors are less common, allowing other error types, such as missing imports, to dominate the relative frequency. In simpler problems, the relative scarcity of these error types inflates the share of missing imports.

Table VII presents the per-model distribution of C++ errors (baseline prompt). We present this analysis for a single language to ensure observed differences are not biased by language-specific error profiles (e.g., “Missing import” errors are more frequent in C/C++).

We observe that error types vary significantly across models. “Missing import” errors are the most frequent category across all models except GPT-oss:120b, which makes only a single such error. There are also differences among the remaining models, with missing imports being particularly dominant for Gemini-2.0-Flash (93.2%). Using two-proportion z-tests over all model pairs with Bonferroni correction and $\alpha=0.05$, GPT-oss:120b and Gemini-2.0-Flash differ significantly from all other models, while most other pairwise differences are not statistically significant. Besides that, smaller models tend to make more “Use undeclared function” and “Incompatible parameter types” errors, both in relative and absolute terms, except for GPT-oss:120b, whose high relative share of “Incompatible parameter types” (29.4%) is not mirrored in absolute terms, as it makes very few errors overall due to rarely making other errors such as missing imports. These differences may stem from variations in models’ training data (e.g., including more LeetCode submissions that lack includes) or training procedures. We also observe that qwen2.5-coder:14b produces substantially fewer compilation errors than its general-purpose

TABLE VII: Compilation error distribution per model (C++) (baseline prompt)

	deepseek-coder-v2:16b	gemini-2.0-flash	gpt-4.1-mini	gpt-oss:120b	qwen2.5-coder:14b	qwen2.5-coder:7b	qwen2.5:14b
Undeclared variable	95 (9.3%)	3 (0.5%)	11 (3.2%)	1 (1.0%)	40 (4.1%)	100 (8.3%)	160 (6.4%)
Use undeclared function	80 (7.9%)	11 (1.9%)	17 (4.9%)	3 (2.9%)	74 (7.6%)	70 (5.8%)	199 (8.0%)
Incompatible param. types	198 (19.4%)	15 (2.6%)	49 (14.1%)	30 (29.4%)	227 (23.4%)	270 (22.3%)	569 (22.9%)
Uninitialized variable	3 (0.3%)	0 (0.0%)	3 (0.9%)	0 (0.0%)	4 (0.4%)	4 (0.3%)	11 (0.4%)
No attribute	29 (2.8%)	1 (0.2%)	9 (2.6%)	8 (7.8%)	34 (3.5%)	71 (5.9%)	86 (3.5%)
Missing import	571 (56.0%)	532 (93.2%)	198 (57.1%)	1 (1.0%)	542 (55.9%)	620 (51.2%)	1331 (53.5%)
Other	12 (1.2%)	3 (0.5%)	14 (4.0%)	15 (14.7%)	10 (1.0%)	21 (1.7%)	32 (1.3%)
Total	1019 (100%)	571 (100%)	347 (100%)	102 (100%)	970 (100%)	1211 (100%)	2486 (100%)

counterpart, while maintaining a similar distribution of error types. This suggests that code-specific fine-tuning primarily reduces the frequency of errors rather than altering their nature. Although analyses for other languages were excluded due to space constraints, these also revealed apparent differences in error distributions across models.

Finally, Table VIII shows the impact of feedback on compilation errors. An error is *fixed* if the code passes all unit tests, *partially fixed* if the error is removed but tests still fail, and *not fixed* if it remains after feedback. We can see that six out of the ten error types, are fixed completely between 12.4% and 15.4% of cases. In contrast, “Undeclared variable” (5.3%), “Incompatible parameter types” (7.8%), and “Use undeclared function” (7.9%) are fixed completely far less often. This gap can be explained by the nature of the fix required: the first group is typically resolved with a single, local edit (e.g., adding an import or initializing a variable), whereas the latter group requires reasoning about how a variable or parameter is used elsewhere in the code, making a complete fix harder even once the error is identified.

Table IX shows the impact of CoT prompting on compilation errors for C++. We present results for a single language to isolate the effect of prompting from language-specific differences, following the same rationale as in the per-model analysis. Analyses for the remaining languages lead to consistent conclusions. Overall, the distribution shifts moderately, but some clear patterns emerge. Most notably, “Missing import” errors become substantially less prevalent (-15.9%), increasing the relative share of all other error types.

B. Runtime Error

Table X presents the per-language (left) and per-problem difficulty (right) distribution of runtime errors as classified by the taxonomy. As in the compilation errors, we can see that only a very small percentage of errors are classified as “Other”, which is a strong indication that our taxonomy covers most of the runtime errors commonly present in LLM-generated code.

TABLE VIII: Impact of feedback on the compilation errors

Error	Not Fixed	Partially fixed	Fixed
Undeclared variable	23.6%	71.0%	5.3%
Use undeclared function	13.7%	78.4%	7.9%
Incompatible param. types	33.7%	58.5%	7.8%
Uninitialized variable	21.8%	62.8%	15.4%
No attribute	15.7%	70.0%	14.3%
Missing import	19.6%	68.0%	12.4%
Missing Public Class	2.2%	83.7%	14.1%
Owner. and life. err.	33.6%	53.5%	12.9%
Trait or type bound error	18.1%	69.0%	13.0%
Other	12.2%	78.2%	9.6%

TABLE IX: Impact of CoT on Compilation Errors (C++)

Undeclared variable	6.8% (+0.7%)
Use undeclared function	12.5% (+5.7%)
Incompatible parameter types	23.7% (+3.4%)
No attribute	6.3% (+2.8%)
Missing import	40.7% (-15.9%)
Other	4.2% (+2.6%)
Total	5909 (-797)

Starting with the per-language distribution, it is evident that, as observed with compilation errors, the types of runtime errors produced by the models show clear language-dependent patterns. The most common categories across all languages are “Out-of-bound” errors, followed by “Incorrect input processing”, “Out-of-memory” errors, and “Recursion” errors. “Out-of-memory” errors are particularly prevalent in C++ (10.3%) and C (33.5%), while they are rare in Java (2.2%) and completely absent in Rust. These differences are statistically significant for all pairwise comparisons (two-proportion z-tests with Bonferroni correction and $\alpha=0.05$). This discrepancy can be attributed to differences in memory management. In C and C++, memory allocation and deallocation are handled manually, and errors such as allocation of excessively large arrays can easily result in runtime errors. In contrast, Java and Rust employ automatic memory management, Java through garbage collection and Rust through its ownership and borrowing system, substantially reducing the likelihood of explicit out-of-memory crashes during execution. Between C and C++, the observed difference can mostly be attributed to the richer standard library and higher-level abstractions available in C++, which enable flexible and incremental memory usage, whereas C exposes only low-level memory primitives, making programs more sensitive to allocation size decisions and out-of-memory failures.

“Out-of-bound” errors are very common across all languages. Although the percentage is higher in C++ (59.9%), C (49.9%), and Java (45.7%) than in Rust (28.2%), this lower percentage does not reflect fewer absolute occurrences. Rather, this is caused by a higher number of errors of other types. Listing 3 shows a Java example. The dynamic programming array *dp* is indexed using *i* - 2 inside the loop without guarding against small values of *i*, meaning that, when *i* = 1, the program attempts to access *dp*[-1], resulting in an error.

```

1 int N = scanner.nextInt();
2 long[][] dp = new long[N + 1][3];
3 for (int i = 0; i <= N; i++) {
4     if (i == 2) { ...
5     } else if (i != 0) {
6         dp[i][0] = (dp[i - 2][0] + dp[i - 2][1] + dp[i - 2][2])
7         ...

```

Listing 3: Example of an out-of-bounds error.

TABLE X: Runtime Error Distribution per language (left) and problem difficulty (right) (baseline prompt)

	C++	Java	C	Rust	Diff. 0	Diff. 1	Diff. 2	Diff. 3
Numeric-overflow	62 (3.8%)	220 (5.4%)	145 (4.9%)	2508 (44.0%)	997 (25.6%)	1153 (20.4%)	559 (15.0%)	226 (21.3%)
Out-of-memory	173 (10.7%)	62 (1.5%)	946 (32.0%)	30 (0.5%)	175 (4.5%)	672 (11.9%)	311 (8.3%)	53 (5.0%)
Undefined-behavior	53 (3.3%)	4 (0.1%)	72 (2.4%)	25 (0.4%)	17 (0.4%)	72 (1.3%)	53 (1.4%)	12 (1.1%)
Out-of-bound	970 (60.0%)	1845 (45.5%)	1510 (51.1%)	1603 (28.1%)	1405 (36.1%)	2256 (40.0%)	1755 (47.0%)	512 (48.3%)
Null pointer dereference	42 (2.6%)	140 (3.5%)	65 (2.2%)	2 (0.0%)	38 (1.0%)	99 (1.8%)	80 (2.1%)	32 (3.0%)
Recursion error	130 (8.0%)	580 (14.3%)	35 (1.2%)	166 (2.9%)	83 (2.1%)	343 (6.1%)	405 (10.8%)	80 (7.6%)
Divided by zero	75 (4.6%)	95 (2.3%)	92 (3.1%)	41 (0.7%)	104 (2.7%)	126 (2.2%)	68 (1.8%)	5 (0.5%)
Incorrect input processing	69 (4.3%)	1045 (25.8%)	36 (1.2%)	1264 (22.2%)	1004 (25.8%)	842 (14.9%)	453 (12.1%)	115 (10.9%)
Total	1616 (100%)	4051 (100%)	2955 (100%)	5700 (100%)	3888 (100%)	5642 (100%)	3733 (100%)	1059 (100%)

TABLE XI: Runtime error distribution per model (Java) (baseline prompt)

	deepseek-coder-v2:16b	gemini-2.0-flash	gpt-4.1-mini	gpt-oss:120b	qwen2.5-coder:14b	qwen2.5-coder:7b	qwen2.5:14b
Numeric-overflow	74 (9.6%)	28 (6.7%)	10 (2.9%)	0 (0.0%)	31 (5.7%)	46 (5.0%)	31 (3.4%)
Out-of-memory	20 (2.6%)	14 (3.4%)	0 (0.0%)	2 (1.5%)	9 (1.6%)	10 (1.1%)	7 (0.8%)
Out-of-bound	343 (44.4%)	168 (40.3%)	136 (39.4%)	55 (41.7%)	248 (45.3%)	435 (47.6%)	460 (49.8%)
Null pointer dereference	16 (2.1%)	10 (2.4%)	7 (2.0%)	16 (12.1%)	19 (3.5%)	22 (2.4%)	50 (5.4%)
Recursion error	111 (14.4%)	118 (28.3%)	103 (29.9%)	28 (21.2%)	63 (11.5%)	72 (7.9%)	85 (9.2%)
Divided by zero	26 (3.4%)	5 (1.2%)	9 (2.6%)	1 (0.8%)	14 (2.6%)	15 (1.6%)	25 (2.7%)
Incorrect input processing	169 (21.9%)	71 (17.0%)	79 (22.9%)	28 (21.2%)	152 (27.7%)	297 (32.5%)	249 (26.9%)
Total	772 (100%)	417 (100%)	345 (100%)	132 (100%)	548 (100%)	913 (100%)	924 (100%)

In contrast, “Incorrect input processing” errors are more frequent in Java (26.0%) and Rust (22.4%) than in C++ (4.3%) or C (1.3%). These differences are statistically significant across all language pairs (two-proportion z-tests with Bonferroni correction and $\alpha=0.05$). This can be explained by differences in input-handling semantics: in C and C++, the generated code frequently relies on input functions such as `cin` and `scanf`, which skip whitespace and do not explicitly recognize empty lines. As a result, certain input format mismatches silently propagate incorrect values, leading to incorrect outputs rather than runtime failures. In contrast, input handling mechanisms in Java and Rust tend to preserve line structure and enforce stricter parsing, causing unhandled input irregularities to surface as runtime errors instead.

Another notable observation is the higher prevalence of “Recursion errors” in Java (14.1%) compared to C++ (8.3%), C (1.2%), and Rust (2.9%). These differences are also statistically significant across all language pairs (two-proportion z-tests with Bonferroni correction and $\alpha=0.05$). These errors typically result from unbounded recursive calls exceeding the call stack limit. The higher frequency in Java can be explained by Java’s comparatively strict default stack size limits.

Finally, “Numeric-overflow” errors appear mostly in Rust (43.9%), with few instances in Java (5.6%), C (4.9%), and C++ (4.0%). This reflects Rust’s runtime overflow checks in debug mode, which detect and report overflow. In C, C++, and Java, overflow occurs silently and may later cause other errors. Listing 4 shows a representative Rust example. The exponentiation `d.pow(n)` is performed without guarding against large values, causing an overflow and triggering a runtime panic. In C, C++, and Java, the same computation would overflow silently and continue with an incorrect value. This silent propagation is a significant dependability and security risk: corrupted intermediate values can affect downstream computations, and, in security-sensitive contexts, be exploited as integer-overflow vulnerabilities. The fact that these faults go undetected in most languages suggests LLM-generated code

may routinely introduce arithmetic errors that are invisible without explicit overflow checks or dedicated static analysis.

When examining problem difficulty, the distribution of runtime error types varies significantly across difficulty levels (chi-square tests with Bonferroni correction and $\alpha=0.05$). “Out-of-bound” errors are the most frequent across all levels, growing from 35.6% in the easiest tasks to 48.4% in the hardest, suggesting that boundary mistakes become more likely as problems become more complex and involve more elaborate data structures or indexing logic. In contrast, “Incorrect input processing” errors are common in easier problems (25.9%), but their relative frequency drops as difficulty rises and the relative frequency of errors of other types increases. A few other trends stand out. “Recursion errors” rise with difficulty, peaking at medium levels (11.1% at difficulty 2), which likely reflects more frequent use of recursive algorithms in problems that require deeper search or divide-and-conquer strategies. Overall, these patterns show that, while simple mistakes dominate early on, higher-difficulty tasks tend to expose structural and logic-related issues that test the models’ ability to manage complexity.

Table XI shows the distribution of Java runtime errors (chosen due to better model performance). Most errors fall into the “Out-of-bound” category, accounting for roughly 40% across all models. Other error types show a more heterogeneous distribution. For instance, “Incorrect input processing” errors are much more frequent in smaller open-source models, occurring around 150–250 times (32.5% in qwen2.5-coder:7b), compared to only 70–80 occurrences in larger proprietary ones (17.0% in Gemini-2.0-Flash and 22.9% in GPT-4.1-mini),

```

1 fn main() {
2     ...
3     let n = parts[0]; % 1000000000000000
4     let p = parts[1]; % 1000000000000000
5     ...
6     if n == 1 { ...
7     } else {
8         for d in (1..p).rev() {
9             if p % d == 0 && (d.pow(n as u32)) <= p {
10                ...

```

Listing 4: Example of a numeric overflow error.

with some of these differences being statistically significant (two-proportion z-tests over all model pairs with Bonferroni correction and $\alpha=0.05$), particularly in comparisons involving one smaller and one bigger model, while most other pairwise differences are not statistically significant. This suggests that larger models are better at understanding user requirements and translating them into correct logic.

Conversely, “Recursion errors” appear more often in larger models, with significantly higher proportions in GPT-4.1-mini (29.9%) and Gemini-2.0-Flash (28.3%) than in smaller models (e.g., 7.9% in qwen2.5-coder:7b). Differences between larger and smaller models are statistically significant (two-proportion z-tests over all model pairs with Bonferroni correction and $\alpha=0.05$), while most comparisons among smaller models are not. This is caused by the lower frequency of other errors, which increases the relative share of recursion-related errors and allows execution to reach deeper recursion levels where errors occur.

Similarly to the trends observed for compilation errors, Qwen2.5-Coder:14b produces fewer runtime errors than its general-purpose counterpart while maintaining a similar distribution of error types. This suggests that code-specific fine-tuning primarily reduces the frequency of runtime errors without substantially altering the underlying types of failures.

Table XII shows the impact of feedback on runtime errors. We label an error as *not fixed* if the updated code still fails to compile or raises a runtime exception, *partially fixed* if it runs but produces incorrect output, and *fixed* if it passes all tests.

Compared to compilation errors, we observe that runtime errors are far less likely to be fully fixed. For instance, “Out-of-memory” remains unsolved in 66.5% of cases, with only 5.1% fully fixed, while “Recursion error” is not solved in 58.9% of cases and only 2.6% fixed. This contrasts with errors like “Missing import”, which appear in compilation errors and are relatively straightforward to correct once identified. The lower fix rates for runtime errors suggest that these issues often involve more complex reasoning about program behavior, such as memory usage, recursion depth, or index handling, which are inherently more difficult for models to resolve completely.

Table XIII shows the impact of CoT prompting on runtime errors for Java. We again focus on a single language to isolate the effect of prompting from language-specific differences. Unlike in compilation errors, the shifts in the runtime error distribution are relatively small: all changes remain below 3.5%, indicating that CoT does not substantially alter the overall profile of runtime failures. This suggests that, runtime errors are less sensitive to prompting style and are more

TABLE XII: Impact of feedback on runtime errors

Error	Not Fixed	Partially Fixed	Fixed
Numeric-overflow	65.2%	27.5%	7.3%
Out-of-memory	70.6%	24.4%	5.0%
Undefined-behavior	75.3%	20.1%	4.5%
Out-of-bound	67.5%	28.1%	4.5%
Null pointer dereference	65.5%	26.9%	7.6%
Recursion error	62.5%	33.9%	3.6%
Divided by zero	65.0%	30.7%	4.3%
Incorrect input processing	64.8%	25.4%	9.9%

TABLE XIII: Impact of CoT on Runtime Errors (Java)

Numeric-overflow	2.6% (-2.8%)
Out-of-memory	3.3% (+1.8%)
Out-of-bound	42.8% (-2.7%)
Null pointer dereference	4.0% (+0.5%)
Recursion error	16.8% (+2.5%)
Divided by zero	1.5% (-0.8%)
Incorrect input processing	26.2% (+0.4%)
Total	4090 (+39)

strongly tied to deeper flaws in program logic.

Still, some patterns emerge. The relative frequency of “Out-of-bound” errors decreases from 45.7% to 42.3%, while “Numeric-overflow” drops from 5.6% to 2.6%. A likely explanation is that CoT prompting encourages more explicit step-by-step reasoning, which may help the model better track array boundaries and arithmetic operations, reducing simple indexing and overflow mistakes. At the same time, “Recursion error” increases from 14.1% to 16.7%, and “Out-of-memory” rises from 1.5% to 3.3%. This may reflect the fact that CoT prompting encourages the generation of more elaborate solutions, which are more likely to rely on recursive decompositions or auxiliary data structures.

C. Underlying Failure Mechanisms

Overall, our results indicate that errors are not isolated, but instead reflect a small number of recurring failure mechanisms related to incomplete program construction, loss of structural consistency as problem complexity increases, and weak reasoning about execution and constraints.

Incomplete program construction. A large fraction of compilation errors reflects the generation of incomplete programs, where required components are missing or not properly integrated. For example, “Missing import” accounts for up to 57.5% of compilation errors in C++ and 36.1% in C, while “Undeclared variable” and “Use of undeclared function” are also consistently observed across languages. These errors suggest limitations in the model’s ability to assemble all necessary elements of a solution and, in some cases, maintain their consistency (e.g., due to scoping issues). Overall, these errors indicate that many failures stem from missing or improperly integrated components rather than incorrect logic.

Structural inconsistency under complexity. As problem difficulty increases, we observe a shift toward errors involving interactions between multiple components, with errors such as “Incompatible parameter types” (compilation), and “Out-of-bound” and “Recursion errors” (runtime) becoming more frequent. In contrast, errors that do not depend on such interactions (e.g., missing imports or incorrect input processing) often decrease. This indicates that higher complexity primarily amplifies failures in coordinating interacting components rather than uniformly increasing all error types.

Execution reasoning and constraint handling. Runtime errors such as “Out-of-bound”, which accounts for up to 59.9% of runtime errors in C++ and remains dominant across all languages, and “Incorrect input processing”, reaching 26.0% in Java and 22.4% in Rust, reflect failures in reasoning about program execution, particularly in handling edge cases, constraints, and input assumptions. Their prevalence across

languages indicates that these are systematic limitations in reasoning about program behavior rather than isolated syntactic issues. As a result, even when code is syntactically correct, models often fail to handle boundary conditions and execution constraints, which are critical for correctness in practice.

Contributing factors. Several factors plausibly explain these patterns. “Incompatible parameter types” and Rust-specific errors likely stem from Rust’s stricter type system and lower training-data representation. “Incorrect input processing” is more common in Java and Rust because their input APIs raise exceptions on format mismatches, whereas `cin/scanf` in C/C++ silently skip whitespace, turning the same logic error into an incorrect output rather than a runtime failure. Structural errors rising with problem difficulty is consistent with longer programs, where consistency across distant parts becomes harder to maintain.

D. Overall trends

Our analysis reveals a set of broad, practical patterns that hold across languages, models, and difficulty levels.

Taxonomy fit. The “Other” category is small and language-specific labels appear where expected, supporting the taxonomy’s usefulness for characterizing compilation and runtime errors across languages.

Error distribution is heavily impacted by programming language and model. The distribution and types of both compilation and runtime errors vary across languages and models, reflecting differences in training data, language semantics, and modeling choices.

All models make basic mistakes. All models frequently generate code with trivial, recurring errors (e.g., missing imports and simple type mistakes), indicating that current models are not reliable for unsupervised code generation.

Errors are concentrated in a small set of classes. In both compilation and runtime errors, most failures fall into few categories, meaning targeted detection and mitigation strategies, such as automated import repair or boundary validation, could yield substantial improvements with minimal effort.

Surface-level errors are fixable; structural errors are not. Iterative feedback resolves errors such as “Missing import” reliably, as these require only local fixes. In contrast, structural errors including “Incompatible parameter types”, “Out-of-bound”, and “Recursion errors” show significantly lower fix rates, reflecting deeper limitations in model reasoning that iterative prompting alone cannot address.

Input handling and boundary logic are disproportionate sources of failure. Out-of-bound errors are the most common runtime failures across all languages, and incorrect input processing accounts for a large share of failures in Java and Rust specifically. Supplying fixed input-handling scaffolding alongside problem descriptions, and systematically validating array bounds and loop conditions in code review, can eliminate entire classes of errors with minimal effort.

High risk of undetected numeric-overflow vulnerabilities. Rust exhibits a substantially higher incidence of numeric-overflow runtime errors than other languages due to its built-

in overflow checks, which trigger panics. Arithmetic faults in unchecked languages execute silently, allowing corrupted computations to propagate undetected. This suggests that arithmetic faults in LLM-generated code may often remain hidden in languages or environments without overflow checks, posing a dependability risk and, in some contexts, a potential security concern when such code is integrated into production systems. More broadly, language choice affects error visibility, as the same generated code may appear correct in one language while containing hidden faults in another, highlighting the need for language-aware validation strategies rather than assuming correctness from the absence of a raised exception.

CoT prompting has an impact on the error distributions

For compilation, it reduces simple errors such as “Missing import”, but increases errors associated with more complex code, including undeclared references and attribute misuse. For runtime errors, CoT is less impactful, with only minor shifts from simpler execution failures toward more structural issues such as recursion and memory-related errors.

V. THREATS TO VALIDITY

Internal validity. Some error categories are underrepresented in the labeled dataset, so we cannot evaluate the classifier’s accuracy for all classes. To address this, we stratified samples by programming language and randomly selected examples within each stratum, ensuring the labeled subset reflects both the language distribution and relative error frequencies in the full dataset. In addition, error classification relies on LLMs, so some labels may be incorrect. To mitigate this, we evaluated performance on 600 manually labeled compilation and runtime samples, confirming strong overall accuracy. A confusion-matrix-based sensitivity analysis, using per-class precision and recall from the evaluation set, confirmed that corrected distributions align closely with observed ones for all dominant classes and do not impact the conclusions.

Construct validity. The taxonomy used in this study includes both language-agnostic and language-specific categories. While this asymmetry is necessary to preserve semantic accuracy, it may affect cross-language comparability. In addition, the analyzed code-generation tasks are self-contained and do not capture the full complexity of real-world software development, such as multi-file dependencies or user interaction. To mitigate these threats, we applied a consistent labeling scheme and uniform classification criteria across all languages and models, ensuring that relative trends remain meaningful despite these limitations. Our analysis focuses on compilation and runtime errors and excludes incorrect-output cases (i.e., programs that execute but produce wrong results) and timeouts. This choice may bias the observed error distribution toward syntactic, type-related, and runtime failure modes, underrepresenting higher-level logical or algorithmic mistakes. We mitigate this risk by explicitly scoping our conclusions to errors that surface through compilation or execution failures, which remain critical for reliability and security in practice.

External validity. The generalizability of the findings is limited by the selection of models and languages. Although the

dataset spans seven LLMs and four programming languages, it does not include the most recent state-of-the-art models. To mitigate these limitations, this work assessed models of various sizes, architectures, and access levels, strengthening confidence that the observed trends are representative of broader LLM behavior. Additionally, the use of competitive-programming-style problems limits generalizability, as these tasks are self-contained and do not capture aspects of real-world development such as multi-file dependencies. This setting may also amplify certain errors, particularly those related to input handling due to strict input formats. However, it provides a controlled environment that isolates fundamental code-generation behavior without confounding factors such as tooling or environment configuration. As such, the results reflect baseline capabilities likely to persist in more complex settings: missing includes affect any IDE-less project, type inconsistencies emerge in long generated code, and boundary errors arise wherever generated code indexes data structures.

VI. CONCLUSIONS

Despite their widespread use and promise for code generation, LLMs still exhibit significant limitations. This paper presents a large-scale empirical study of errors in LLM-generated code, characterizing the types and prevalence of errors across models and languages. We analyze 86,726 compilation and runtime errors produced by seven LLMs in four programming languages, examining how error behavior varies with language, problem difficulty, and iterative feedback.

Our results show that error patterns differ substantially across models and languages, reflecting differences in training data and language-specific challenges. Even bigger commercial models frequently produce simple but consequential mistakes. We repeatedly observe failures linked to weak input handling, missing boundary checks, unsafe memory assumptions, and arithmetic faults. While some languages detect these issues at runtime, others fail silently, increasing the risk of undetected security-relevant problems. Overall, although our analysis does not cover incorrect outputs, these findings suggest current LLMs cannot reliably produce secure, correct code without human oversight, highlighting the need for rigorous validation and review in AI-assisted development.

As future work, we plan to extend our analysis to additional languages and models, and to incorporate more complex tasks involving multi-file dependencies, external libraries, and larger codebases to better reflect real-world scenarios. We also aim to apply the same analysis to incorrect outputs and timeout errors, and investigate why certain error types persist under iterative feedback and how models modify code during refinement. We expect this work to provide a comprehensive view of the limitations of LLM-generated code and to lay a strong foundation for targeted mitigation strategies, improved prompting techniques, and more robust code-generation models.

REFERENCES

- [1] (2025) Satya nadella says as much as 30% of microsoft code is written by ai. <https://web.archive.org/web/20250430014530/https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html>. Accessed: 2025-05-05.

- [2] S. Perkel, "Microsoft cto breaks down how he sees software developer jobs evolving in the next 5 years," *Business Insider*, April 2025, <https://www.businessinsider.com/microsoft-cto-ai-generated-code-software-developer-job-change-2025-4>.
- [3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of ai on developer productivity: Evidence from github copilot," 2023.
- [4] M. I. H. Shihab, C. Hundhausen, A. Tariq, S. Haque, Y. Qiao, and B. W. Mulanda, "The effects of github copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks," in *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1*, ser. ICER '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 407–420.
- [5] D. Hendrycks *et al.*, "Measuring coding challenge competence with apps," 2021.
- [6] R. P. Nogueira, M. Vieira, and J. R. Campos, "Beyond functional correctness: An empirical evaluation of large language models for text-to-code generation," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 264–275.
- [7] F. Liu *et al.*, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv preprint arXiv:2404.00971*, 2024.
- [8] Z. Wang *et al.*, "Towards understanding the characteristics of code generation errors made by large language models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 717–717.
- [9] D. Song *et al.*, "An empirical study of code generation errors made by large language models," in *7th Annual Symposium on Machine Programming*, 2023.
- [10] F. Tambon, A. Moradi-Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, and G. Antoniol, "Bugs in large language models generated code: An empirical study," *Empirical Software Engineering*, vol. 30, no. 3, p. 65, 2025.
- [11] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? assessing the quality of code generation by chatgpt," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, 2024.
- [12] Q. Chen, J. Yu, J. Li, J. Deng, J. T. J. Chen, and I. Ahmed, "A deep dive into large language model code generation mistakes: What and why?" *arXiv preprint arXiv:2411.01414*, 2024.
- [13] R. Mo *et al.*, "Assessing and analyzing the correctness of github copilot's code suggestions," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 7, pp. 1–32, 2025.
- [14] OpenAI, "Gpt-4 technical report," 2024.
- [15] A. Liu *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [16] Q. Zhu *et al.*, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024.
- [17] B. Hui *et al.*, "Qwen2.5-coder technical report," 2024.
- [18] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," 2023.
- [19] M. Chen *et al.*, "Evaluating large language models trained on code," 2021.
- [20] B. Ray, D. Posnett, V. Filkov, and P. Devanbu, "A large scale study of programming languages and code quality in github," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 155–165.
- [21] R. Chillarege *et al.*, "Orthogonal defect classification-a concept for in-process measurements," *IEEE Transactions on software Engineering*, vol. 18, no. 11, pp. 943–956, 1992.
- [22] J. Austin *et al.*, "Program synthesis with large language models," 2021.
- [23] S. Lu *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," 2021.
- [24] R. P. Nogueira, M. Vieira, and J. R. Campos, "Probe: Benchmarking code generation in large language models," 2026. [Online]. Available: <https://arxiv.org/abs/2607.13820>
- [25] R. Puri *et al.*, "Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks," 2021.
- [26] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [27] M. Vieira, P. A. Shah, B. Shah, and R. Krasniqi, "Polyglot: An extensible framework to benchmark code translation with llms," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 2363–2375.